EMBEDDED SYSTEMS • COMMUNICATION PROTOCOLS

I²C PROTOCOL

Inter-Integrated Circuit — Complete Structured Notes

From protocol fundamentals to STM32F407x driver-level hardware — signals, timing, addressing, modes, and peripheral registers.

SDA / SCL Signals | START / STOP | ACK / NACK | Repeated START | I2C Modes | STM32 Peripheral

Table of Contents

SECTION 01

I²C Protocol Basics & I²C vs SPI

What is I²C?

I²C (Inter-Integrated Circuit), pronounced "I-squared-C" or "I-two-C", is a serial communication protocol used between integrated circuits located close together on the same PCB. Unlike SPI, I²C follows a **standardized specification** developed by **NXP Semiconductors**, which makes communication between devices from different manufacturers reliable and interoperable.

WHY I²C IS MORE COMPLEX THAN SPI

I²C defines rules for: data transmission, data reception, handshaking, error handling, arbitration between masters, acknowledgement, and clock stretching. In short — **SPI = simple protocol**, **I²C = standardized protocol with many built-in features.**

Key Features of I²C

- Serial, half-duplex communication
- Only 2 wires required (SDA & SCL)
- Address-based communication (no chip-select pins)
- Supports multiple masters, with automatic hardware arbitration
- Automatic acknowledgements (ACK/NACK)
- Clock stretching support

I²C vs SPI — Comparison Table

Feature	I²C	SPI
Specification	Standard spec by NXP	No universal standard
Number of wires	2	4 or more
Multi-master	Supported	Not defined in protocol
Arbitration	Automatic (hardware)	Software responsibility
ACK / NACK	Automatic	Not supported
Communication type	Half duplex	Full duplex
Addressing	Device addresses	Slave Select (SS) pin
Clock stretching	Supported	Not supported
Maximum speed	Up to 4 MHz	Peripheral Clock ÷ 2 (much higher)
Data rate	Low	Very high

Detailed Differences

1. Pins Required

I²C uses only **SDA** (Serial Data) and **SCL** (Serial Clock) — shared by every device on the bus. SPI requires MOSI, MISO, SCLK, and one **Slave Select (SS)** line *per slave*. For 1 master + 3 slaves, SPI needs 6 pins vs I²C's fixed 2 pins.

Configuration	Pins Required
SPI (1 master + 3 slaves)	6 pins (MOSI, MISO, SCLK, SS1–3)
I²C (any number of slaves)	Only 2 pins (SDA, SCL)

2. Multi-Master Capability

I²C natively supports multiple masters. If two masters attempt to transmit simultaneously, dedicated hardware performs **arbitration** automatically — no software intervention needed. SPI does not define multi-master operation in the protocol itself; if supported by a microcontroller, arbitration must be handled in software.

3. Automatic Acknowledgement

Every byte on I²C is automatically acknowledged by hardware (ACK/NACK). SPI has no such mechanism — acknowledgement must be implemented manually if required.

4. Addressing

Every I²C slave has a unique address; the master selects a slave by transmitting that address. SPI instead uses a dedicated Slave Select (SS/CS) pin per device.

5. Duplex Mode

I²C is **half-duplex** — communication occurs in only one direction at a time. SPI is **full-duplex** — both devices can transmit and receive simultaneously.

6. Speed

I²C's theoretical maximum is 4 MHz (Ultra Fast Mode), though most microcontrollers (e.g. STM32) only support 100 kHz–1 MHz. SPI speed = Peripheral Clock ÷ 2, e.g. a 40 MHz peripheral clock yields 20 MHz SPI — roughly **50× faster** than typical I²C (400 kbps).

Protocol	Maximum Data Rate (example: 40 MHz peripheral clock)
I²C	400 kbps
SPI	20 Mbps

7. Clock Stretching

If an I²C slave is busy, it can hold SCL LOW, forcing the master to wait — this is **clock stretching**, and it prevents data loss. SPI slaves cannot stretch or stop the master's clock.

Suitable Applications

I²C is best for:

- Temperature / humidity / pressure sensors
- EEPROM
- Real-Time Clock (RTC)
- Accelerometers
- Small displays & configuration registers

SPI is best for:

- LCD displays
- SD Cards
- Audio streaming
- Flash memory
- High-speed ADC/DAC, camera modules

Important Terminology

Term	Meaning
Transmitter	Device sending data onto the bus
Receiver	Device receiving data from the bus
Master	Initiates communication, generates clock, controls & terminates the bus
Slave	Addressed device; never initiates communication
Multi-Master	A bus on which more than one device can act as master
Arbitration	Hardware process deciding which master continues when two start simultaneously

Key Takeaways

- I²C is a standardized, 2-wire, half-duplex protocol with built-in addressing, ACK, and arbitration.
- SPI is faster and full-duplex but requires more pins and has no universal standard.
- Choose I²C for low-speed, many-device sensor buses; choose SPI for high-speed, point-heavy links.

SECTION 02

SDA & SCL Signal Characteristics

The Two Bus Lines

Signal	Full Form	Purpose
SDA	Serial Data Line	Transfers data between master and slave
SCL	Serial Clock Line	Carries the clock generated by the master

Both lines are shared by every device on the bus.

Important Characteristics

- Bidirectional communication lines
- Connected to VDD through **pull-up resistors**
- Shared among all masters and slaves
- Must use **Open Drain** (or Open Collector) output configuration
- When idle, both lines remain **HIGH**

BIDIRECTIONAL NATURE

Unlike SPI — where MOSI and MISO have fixed directions — in I²C, SDA carries data in both directions, and SCL can even be influenced by a slave during clock stretching.

Pull-Up Resistors

Both SDA and SCL connect to VDD through pull-up resistors. They keep the lines HIGH when idle, let devices pull the line LOW safely, and allow multiple devices to share the bus without conflict.

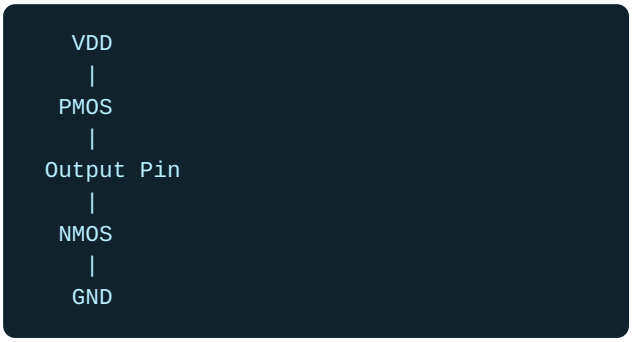
BUS IDLE CONDITION

When no communication is occurring: **SDA = HIGH** and **SCL = HIGH**. This is the "Idle Bus State."

Open Drain Output Requirement

The I²C specification mandates that **all I²C output pins be configured as Open Drain** (or Open Collector). In a normal Push-Pull GPIO, a PMOS drives the pin HIGH and an NMOS drives it LOW. In Open Drain, the PMOS is removed — only the NMOS remains, so the pin can pull LOW but cannot drive HIGH on its own; a pull-up resistor is required to achieve the HIGH state.

Push-Pull (normal GPIO)



Open Drain (I²C)

WHY OPEN DRAIN?

It allows multiple devices to share SDA/SCL safely — any device can pull the bus LOW with no electrical conflict. Without Open Drain, two devices driving opposite logic levels simultaneously could cause bus contention and hardware damage.

GPIO Configuration Summary for I²C

Setting	Value
Mode	Alternate Function
Output Type	Open Drain
Pull-up	Internal or External
Speed	As required

Internal vs External Pull-Up

Type	Advantages	Disadvantages
Internal Pull-up	No external components, simple design	Usually weak, unsuitable for high speed
External Pull-up	Stronger, better signal quality, preferred in practice	Requires extra component

The correct pull-up resistor value depends on bus capacitance, supply voltage, and communication speed, calculated using formulas from the I²C specification. Bus capacitance also **limits the number of devices** that can be reliably connected — more devices increase capacitance and slow the signal rise time.

Troubleshooting Checklist

STEP-BY-STEP DEBUG

After initializing the I²C peripheral, measure voltage between SDA–GND and SCL–GND. Both should read ≈ VDD (e.g. 3.3 V). If not HIGH, check for: missing pull-up resistor, incorrect GPIO configuration, Push-Pull mode used instead of Open Drain, faulty wiring, short circuit, or a device holding the bus LOW.

- GPIO mode set to Alternate Function
- Output type is Open Drain
- Pull-up resistor present (internal or external)
- SDA is HIGH when idle
- SCL is HIGH when idle
- Supply voltage correct
- Wiring correct; no device permanently pulling the bus LOW

Key Takeaways

- SDA and SCL are bidirectional, Open Drain, pulled HIGH via resistors, and idle HIGH.
- Open Drain output prevents bus contention among multiple devices.
- Bus capacitance limits maximum device count and speed.
- A quick voltage check on SDA/SCL is the first troubleshooting step for I²C faults.

SECTION 03

I²C Communication Modes

I²C defines multiple modes, each with a different maximum data rate. The chosen mode affects speed, compatibility, and driver/timing configuration.

Modes Overview

Mode	Max Data Rate	STM32F4 Support	Typical Usage
Standard Mode (Sm)	100 kbps	Almost all STM32 MCUs	Basic sensors, RTCs, EEPROMs
Fast Mode (Fm)	400 kbps	Almost all STM32F4 MCUs	Faster sensors, displays
Fast Mode Plus (Fm+)	1 Mbps	Only some STM32F4 MCUs	High-performance peripherals
High-Speed Mode (Hs)	3.4 Mbps	Not supported on STM32F4	Very high-speed I²C

Always verify Fast Mode Plus / High-Speed Mode support against the Reference Manual and datasheet of the specific MCU.

Standard Mode (Sm) — 100 kbps

The original I²C mode, supported by almost every I²C-enabled microcontroller. Suitable for low-throughput peripherals: RTCs, temperature/humidity/pressure sensors, EEPROM.

COMPATIBILITY RULE

Standard Mode devices are **NOT upward compatible**. A device that only supports 100 kbps cannot reliably communicate on a bus running Fast Mode (400 kbps) or higher.

Fast Mode (Fm) — 400 kbps

Roughly 4× faster than Standard Mode; supported by nearly all STM32F4 MCUs.

COMPATIBILITY RULE

Fast Mode devices **are downward compatible** — they can operate on a Standard Mode (100 kbps) bus at the lower speed. However, Standard-Mode-only devices must never be placed on a 400 kbps bus.

Fast Mode Plus (Fm+) — 1 Mbps

Faster than Fast Mode; supported only by **selected** STM32F4 microcontrollers. Always confirm support in the Reference Manual before use.

High-Speed Mode (Hs) — 3.4 Mbps

Designed for very high-speed I²C; **not supported** by the STM32F4 family. Support on other STM32 families (F3, F7, etc.) varies by device.

Compatibility Summary

Device Type	100 kbps Bus	400 kbps Bus
Standard Mode Device	✔ Works	✘ Not Supported
Fast Mode Device	✔ Works	✔ Works

Practical Selection Guidelines

Application	Recommended Mode
RTC / Temperature / EEPROM / Humidity sensor	Standard Mode
Moderate-speed sensors / faster displays	Fast Mode
High-performance devices	Fast Mode Plus (if supported)

DRIVER DEVELOPMENT NOTE

When writing an I²C driver, the operating mode should be a **configurable parameter** (e.g. Standard or Fast Mode), and the driver must configure the peripheral's timing/clock registers accordingly. This course covers only Standard Mode (100 kbps) and Fast Mode (400 kbps).

Key Takeaways

- Four modes exist: Standard (100k), Fast (400k), Fast+ (1M), High-Speed (3.4M).
- Standard Mode devices are not upward compatible; Fast Mode devices are downward compatible.
- STM32F4 does not support High-Speed Mode; Fast Mode Plus support is MCU-specific.
- Communication mode should be a configurable driver parameter.

SECTION 04

Protocol Basics — Frame Structure

A complete I²C transaction follows a fixed sequence, always initiated and terminated by the master. The slave never initiates communication.

Basic Communication Flow

```
START → Address + R/W → ACK → Data Byte → ACK → Data Byte → ACK → STOP
```

Step 1 – START Condition

Generated by the master. It signals the beginning of communication, gives the master control of the bus, and alerts all slaves to listen for the upcoming address.

Step 2 – Address Phase (always 8 bits)

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
A6	A5	A4	A3	A2	A1	A0	R/W

The first 7 bits identify the target slave (unique per device); the 8th bit is the Read/Write flag.

R/W Bit	Meaning
0	Master **writes** data to the slave
1	Master **reads** data from the slave

*Memory tip: **0 = Write to Slave, 1 = Read from Slave***

> **BYTE-ORIENTED RULE**
>
> Every transmission on SDA must be exactly 8 bits — never 7, 9, or 16. Each transfer is one full byte, always followed by an ACK/NACK bit.

Write Operation (R/W = 0)

```
START → Address+Write(0) → Slave ACK → Data Byte → Slave ACK → Data Byte → Slave ACK → STOP
```

1. Master generates START
2. Master sends 7-bit address + Write bit (0)
3. Addressed slave checks the address and sends ACK
4. Master sends one data byte; slave ACKs

5. Repeat for additional bytes
6. Master generates STOP

Read Operation (R/W = 1)

```
START → Address+Read(1) → Slave ACK → Slave Data → Master ACK → Slave Data → Master ACK → STOP
```

1. Master generates START
2. Master sends address + Read bit (1)
3. Slave checks address and sends ACK
4. Slave transmits a data byte; master ACKs
5. Repeat as needed
6. Master generates STOP when finished

Who Sends the ACK?

Communication	Who Sends ACK?
Address sent by Master	Slave
Data written by Master	Slave
Data sent by Slave (Read)	Master

Data Order & STOP

I²C transmits data **MSB first**. The STOP condition (generated by the master) releases the bus, indicating communication has finished and allowing another master to use it.

Key Takeaways

- Only the master initiates and terminates communication.
- Every transaction: START → Address Phase (7-bit address + R/W) → ACK → Data (byte-oriented, each followed by ACK) → STOP.
- R/W = 0 is Write (Master→Slave); R/W = 1 is Read (Slave→Master).
- Data is transmitted MSB first.

SECTION 05

START & STOP Conditions

Every I²C transaction is bounded by two special conditions, both generated by the master: **START (S)** and **STOP (P)**.

```
START (S) → Address Phase → ACK → Data Transfer → ACK(s) → STOP (P)
```

START Condition (S)

Generated when **SDA transitions HIGH → LOW while SCL remains HIGH**. This signals the beginning of a transaction, marks the bus as busy, and alerts all devices to listen for the address.

STOP Condition (P)

Generated when **SDA transitions LOW → HIGH while SCL remains HIGH**. It ends the transaction, releases the bus, and returns SDA/SCL to their idle HIGH state.

START vs STOP Summary

Feature	START (S)	STOP (P)
SDA Transition	HIGH → LOW	LOW → HIGH
SCL State	HIGH	HIGH
Generated By	Master	Master
Purpose	Begin communication	End communication
Bus State	Busy	Free

INTERVIEW TIP

A frequently asked question: define START and STOP. Remember — it's the direction of the SDA transition *while SCL is HIGH* that matters.

Repeated START (Sr)

Sometimes the master must continue communication without releasing the bus. Instead of STOP followed by a new START, it issues another START directly — this is a **Repeated START**.

```
START → Address → ACK → Data → ACK → Repeated START → Address → ACK → Read/Write continues
```

- Generated by the master; no STOP occurs before it
- The bus remains busy and ownership stays with the same master
- Commonly used when writing a register address, then immediately reading data from it

Common Interview Questions

Question	Answer
What defines a START condition?	SDA transitions HIGH→LOW while SCL is HIGH
What defines a STOP condition?	SDA transitions LOW→HIGH while SCL is HIGH
Who generates START/STOP?	The master
When is the bus busy?	Immediately after START
When is the bus free?	After STOP (following bus-free time)

Key Takeaways

- START: SDA HIGH→LOW while SCL HIGH. STOP: SDA LOW→HIGH while SCL HIGH.
- Bus is busy after START, free after STOP.
- Repeated START keeps the bus busy, avoiding release between related operations.
- Most MCU I²C peripherals auto-assume Master role on START and release it on STOP.

SECTION 06

Address Phase, ACK & NACK

Address Phase Structure (1 byte)

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
A6	A5	A4	A3	A2	A1	A0	R/W

```
Clock : 1 2 3 4 5 6 7 8
        | | | | | | | |
Data  : A A A A A A A R/W

Clocks 1–7: Slave Address     Clock 8: Read/Write bit
```

Each slave compares the received address to its own; a match triggers an ACK response, otherwise the slave ignores the transmission.

Acknowledgement (ACK) Mechanism

After every transmitted byte, the 9th clock pulse is reserved for ACK/NACK. The transmitter releases SDA during this clock; the receiver then controls the line.

SDA during 9th Clock	Meaning
LOW	**ACK** (Acknowledge)
HIGH	**NACK** (Not Acknowledge)

```
Clock : 1 2 3 4 5 6 7 8 9
        | | | | | | | | |
Data  : A A A A A A A R ACK
```

> **CLOCK GENERATION DURING ACK**
>
> Even during the ACK cycle, the **master always generates the clock** — including the 9th pulse. The receiver only controls the SDA line during that pulse.

Who Sends ACK?

Transmitted Byte	Receiver	ACK Sent By
Address (from Master)	Slave	Slave
Data (Master → Slave)	Slave	Slave
Data (Slave → Master)	Master	Master

Why NACK Happens & What Follows

Common causes of a NACK: no slave responded to the address, receiver unable/unwilling to accept more data, a communication error, or (during a read) the master has received all data it needs.

```
NACK → STOP (end communication)  or  Repeated START (begin new transaction)
```

Quick Revision Table

Item	Description
Address Phase Length	8 bits (1 byte)
Slave Address	First 7 bits
R/W Bit	8th bit
R/W = 0	Write to Slave
R/W = 1	Read from Slave
ACK/NACK Position	9th clock pulse
ACK	SDA = LOW
NACK	SDA = HIGH
ACK Generated By	Receiver
Clock During ACK	Generated by Master
After NACK	STOP or Repeated START

Key Takeaways

- The Address Phase always follows START and is exactly 8 bits.
- Every 8-bit byte is followed by an ACK/NACK on the 9th clock pulse.
- ACK = SDA LOW; NACK = SDA HIGH, both sampled while the master drives SCL HIGH.
- After a NACK, the master issues either STOP or Repeated START.

SECTION 07

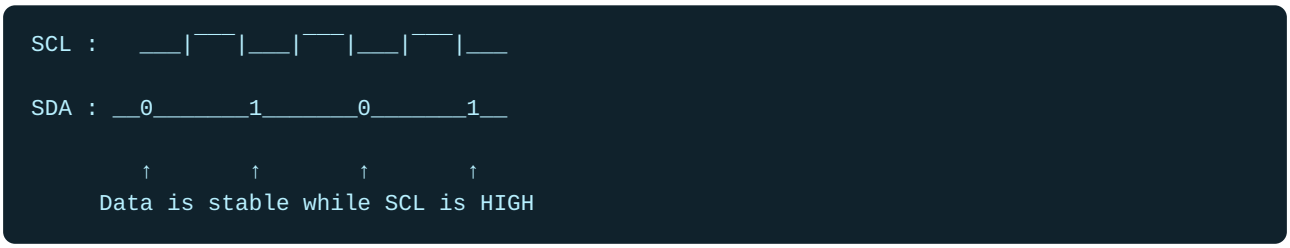
Data Validity Rule

GOLDEN RULE OF I²C

SDA can change only when SCL is LOW. Equivalently: data must be stable whenever SCL is HIGH. The receiver samples data only during the HIGH period of SCL.

Why This Rule Exists

If data changed while the clock was HIGH, the receiver couldn't reliably tell whether the bit is 0, 1, or transitioning — causing incorrect reception. Therefore: **Clock HIGH → Read Data**, **Clock LOW → Change Data**.



Exceptions: START and STOP

SDA is allowed to change while SCL is HIGH **only** during the START and STOP conditions.

SDA Transition	SCL State	Meaning
Changes	LOW	Data Bit
HIGH → LOW	HIGH	START Condition
LOW → HIGH	HIGH	STOP Condition

This simple rule lets the receiver reliably distinguish data bits from control signals.

MEMORY TRICK

"High = Hold, Low = Change" — SCL HIGH holds the data stable; SCL LOW allows the data to change.

Common Interview Questions

Q: Why is data allowed to change only when SCL is LOW?

Because the receiver reads data during the HIGH period of SCL; keeping data stable during that window ensures reliable sampling and prevents incorrect bit detection.

Q: Why are START/STOP generated while SCL is HIGH?

> Since data never changes while SCL is HIGH, any SDA transition during that window cannot be a data bit — so the receiver correctly interprets it as a START or STOP control signal.

Key Takeaways

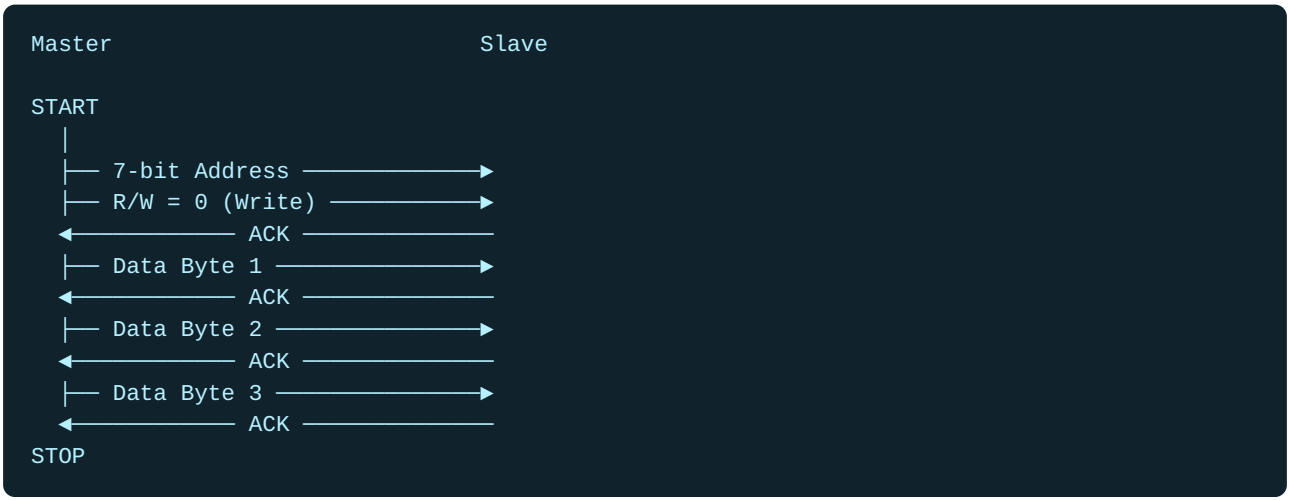
- Data must be stable while SCL is HIGH; SDA may only change while SCL is LOW.
- START/STOP are the sole exceptions, both occurring while SCL is HIGH.
- This rule is one of the most frequently asked I²C interview topics.

SECTION 08

Master Write & Master Read Sequences

1. Master Writing Data to Slave (R/W = 0)

Master is the transmitter; slave is the receiver.



Step	Sender	Receiver	Data
1	Master	All Slaves	START
2	Master	Slave	7-bit Address
3	Master	Slave	R/W = 0
4	Slave	Master	ACK
5–10	Master / Slave (alternating)	—	Data Byte / ACK × 3
11	Master	All	STOP

2. Master Reading Data from Slave (R/W = 1)

Roles reverse: the slave becomes the transmitter, the master becomes the receiver.

> **WHY DOES THE MASTER SEND ACK WHILE READING?**
>
> The slave doesn't inherently know how many bytes the master wants. **ACK** = "continue sending more bytes." **NACK** (sent after the final byte) = "I have enough data, stop transmitting" — followed by STOP.

Write vs Read — Side by Side

Feature	Master Write	Master Read
R/W Bit	0	1
Data Sender	Master	Slave
Data Receiver	Slave	Master
Who ACKs each byte?	Slave	Master
Communication Ends	Master sends STOP	Master sends NACK, then STOP

MASTER WRITE

```
START → Address → R/W=0 → ACK → Data1 →
ACK → Data2 → ACK → Data3 → ACK → STOP
```

MASTER READ

```
START → Address → R/W=1 → ACK → Data1 →
ACK → Data2 → ACK → Data3 → NACK → STOP
```

Interview Questions

Question	Answer
How does the slave know the master wants to write/read?	The R/W bit: 0 = write, 1 = read
Why does the master send ACK during a read?	To tell the slave the byte was received correctly and more data is wanted
Why NACK after the last byte in a read?	To tell the slave to stop transmitting further bytes
Who generates STOP?	Always the master

Key Takeaways

- Write: Master sends data, Slave ACKs each byte, Master ends with STOP.
- Read: Slave sends data, Master ACKs each byte except the last (NACK), then STOP.
- Data is always MSB first, byte-oriented, ACK/NACK after every byte.

SECTION 09

Repeated START (Sr)

A **Repeated START (Sr)** is a START condition generated **without** first issuing a STOP. In short: *Repeated START = START again, without STOP.*

Why It's Needed

Many transactions must perform a Write followed immediately by a Read (or vice versa) as a single continuous operation — e.g. writing a register/memory address, then reading its contents. If a STOP is issued between the two, the bus is released and, on a multi-master system, another master could take over before the transaction completes. Repeated START keeps the same master in control throughout.

Example Scenario: Reading an EEPROM Register

Goal: read the data stored at EEPROM memory address **0x45**. Since the EEPROM doesn't know which location to read, the master must first write the address, then read the data — two operations combined.

Method 1 – Without Repeated START (two separate transactions)

```
START
  |
Address + Write (R/W=0)
  |
ACK
  |
Memory Address = 0x45
  |
ACK
  |
STOP
──────────────────────────────
START
  |
Address + Read (R/W=1)
  |
ACK
  |
Data at 0x45
  |
NACK
  |
STOP
```

PROBLEM

After the first STOP, the bus is released before the actual read completes. On a multi-master bus, another master could take control before the read begins, interrupting the intended operation.

Method 2 – Using Repeated START (one continuous transaction)

```
START
   |
Address + Write (R/W=0)
   |
ACK
   |
Memory Address = 0x45
   |
ACK
   |
Repeated START (Sr)
   |
Address + Read (R/W=1)
   |
ACK
   |
Data at 0x45
   |
NACK
   |
STOP
```

The bus stays busy and under the same master's control the entire time — no other master can interrupt.

Bus Ownership Comparison

Without Repeated START	With Repeated START
STOP generated between operations	No STOP between operations
Bus is released	Bus remains occupied
Another master may take control	No other master can interrupt
Two separate transactions	One continuous transaction
Suitable for single-master systems	Preferred for multi-master & register-based devices

When Is It Used?

- Reading data from an EEPROM
- Reading a sensor register (write register address, then read data)
- Accessing configuration registers of peripherals

SINGLE-MASTER VS MULTI-MASTER

On a multi-master bus, Repeated START is strongly recommended. On a single-master bus, either STOP+START or Repeated START works correctly — but Repeated START is still commonly used since many I²C devices expect this sequence.

Interview Questions

Question	Answer
What is Repeated START?	A START issued without a preceding STOP, letting the master continue without releasing the bus
Why use it?	Prevents the bus being released between related operations, avoiding interruption by another master
When is it typically used?	Combined Write→Read or Read→Write operations, e.g. EEPROM/register access
Why does an EEPROM need a Write before Read?	So it knows which memory location's data to return

MEMORY TRICK

"STOP releases the bus, Repeated START keeps the bus." | START → Write → Sr → Read → STOP

Key Takeaways

- Repeated START = new START without a preceding STOP; bus ownership is retained.
- Essential for atomic Write→Read sequences like EEPROM/register access.
- Critical on multi-master buses to prevent another master interrupting the transaction.

SECTION 10

STM32F407x I²C Peripheral Overview

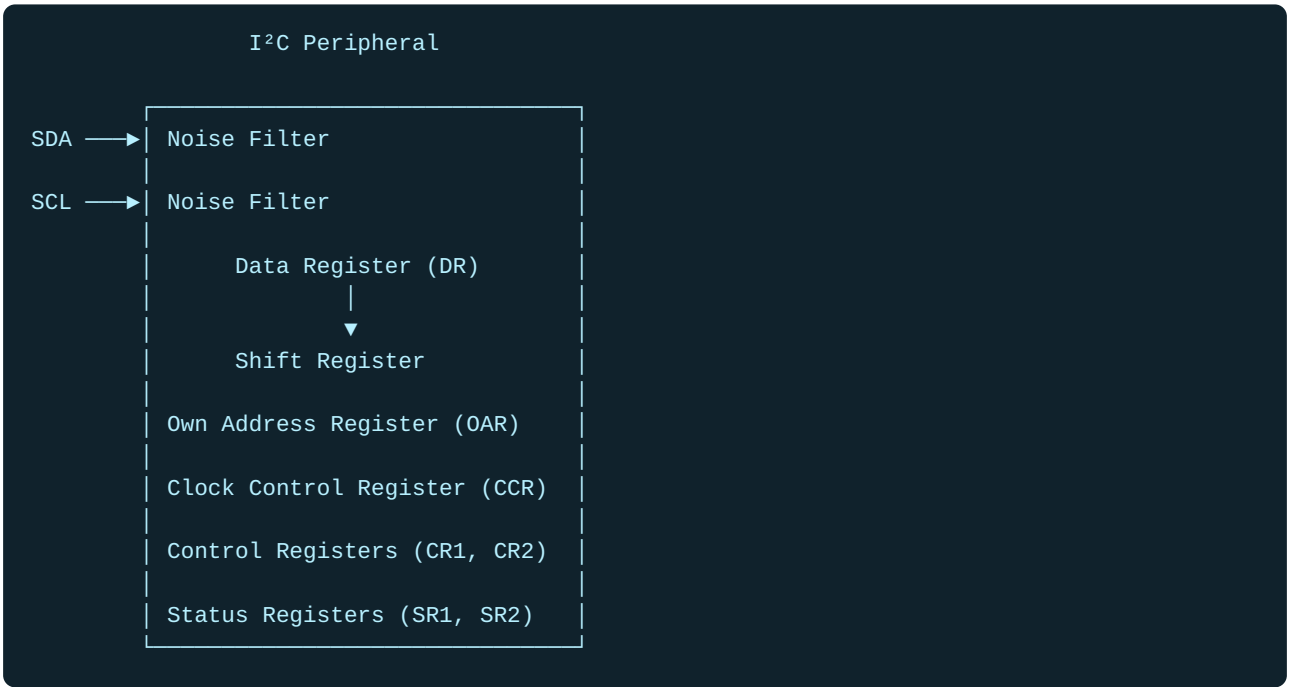
Identifying the I²C Peripherals

Driver development begins by studying the microcontroller's Reference Manual to determine how many I²C peripherals exist, which bus they're on, and their associated registers.

Peripheral	Connected Bus
I²C1	APB1
I²C2	APB1
I²C3	APB1

The STM32F407x has 3 I²C peripherals, all on APB1. Other STM32 devices may differ — always check the Reference Manual.

I²C Peripheral Block Diagram



Major Components

1. SDA & SCL Pins with Noise Filters

The only two pins required for normal operation. Both pass through a **noise filter** before reaching internal logic, removing glitches, spikes, and electrical noise — improving reliability and preventing false START/STOP detection.

STM32 I²C peripherals also support SMBus, but it is not covered in this course.

2. Data Register (DR)

Only **one** Data Register exists, because I²C is half-duplex — TX and RX never happen simultaneously, so separate registers aren't needed.

Transmission

```
CPU → DR → Shift Register → SDA Pin
```

Reception

```
SDA Pin → Shift Register → DR → CPU reads
```

3. Shift Register

Converts 8-bit parallel data to a serial bit stream during transmission, and serial bits back to 8-bit parallel data during reception — the interface between DR and the external SDA line.

4. Own Address Register (OAR)

Stores the microcontroller's own slave address — used **only in Slave Mode**. When another master sends this address, the peripheral recognizes it and responds. Not required in Master Mode, since a master does not need its own address.

5. Clock Control Register (CCR)

Configures the frequency of SCL (e.g. 100 kHz Standard Mode, 400 kHz Fast Mode). The firmware sets CCR to achieve the desired I²C clock speed.

6. Control Registers (CR1, CR2)

Register	Purpose
CR1	Enables/disables the peripheral; controls core operating features
CR2	Configures additional parameters such as clock frequency and interrupts

7. Status Registers (SR1, SR2)

Register	Purpose
SR1	Communication event/status flags
SR2	Additional status information

Examples of status info: START generated, address matched, data transmitted/received, ACK received, bus busy. The driver polls these flags to track communication progress.

Data Flow Inside the Peripheral

Transmit

```
CPU → DR → Shift Register → SDA Pin → Slave Device
```

Receive

```
Slave Device → SDA Pin → Shift Register → DR → CPU
```

Master Mode vs Slave Mode

Feature	Master Mode	Slave Mode
Initiates communication	Yes	No
Generates SCL	Yes	No
Needs Own Address Register	No	Yes
Uses CCR to generate clock	Yes	No (clock supplied by master)

Register Summary

Register	Full Form	Function
DR	Data Register	Stores transmitted or received data
OAR	Own Address Register	Stores the device's slave address
CCR	Clock Control Register	Configures SCL frequency
CR1	Control Register 1	Enables/configures the peripheral
CR2	Control Register 2	Additional configuration options
SR1	Status Register 1	Communication status flags
SR2	Status Register 2	Additional communication status

MEMORY TRICK — REGISTER ORDER

DR → OAR → CCR → CR1/CR2 → SR1/SR2 (Data → Own Address → Clock Control → Configure → Status)

Interview Questions

Question	Answer
How many I²C peripherals in STM32F407x?	Three (I²C1, I²C2, I²C3), all on APB1
Why only one Data Register?	I²C is half-duplex, so TX/RX never need separate registers

Function of the Own Address Register?	Stores the MCU's slave address; used only in slave mode
Function of the CCR?	Configures SCL clock frequency (e.g. 100 kHz, 400 kHz)
Why noise filters on SDA/SCL?	Remove glitches/spikes/noise, improving reliability and preventing false signal detection

Key Takeaways

- STM32F407x has 3 I²C peripherals (I²C1–3), all connected to APB1.
- Only SDA and SCL are needed externally; both pass through noise filters.
- A single Data Register + Shift Register handles half-duplex serial transfer.
- OAR is used only in slave mode; CCR sets SCL frequency; CR1/CR2 configure the peripheral; SR1/SR2 report status.